



Omics Unlayered
 662 subscribers

Subscribe

OMICS UNLAYERED

ISSUE 01

What your pipeline *assumes* but never checks

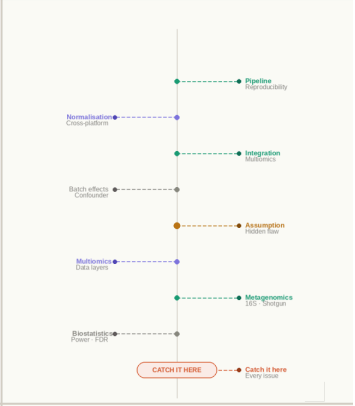
One analytical failure per issue. What it costs labs that miss it — and what to do instead.

Pipeline

Multimics

Metagenomics

Bioinformatics



OMICS UNLAYERED

Parth Doshi

Bioinformatics Consultant

Issue 1 - Why Bioinformatics Pipelines Fail Six Months Later



Parth Doshi


Bioinformatics Consultant | I build the analytical infrastructure research labs need — pipelines, multimics...




May 11, 2026

The 2026 reproducibility stack most labs still aren't using

A Bioinformatics pipeline can run perfectly today and still fail six months from now.

Not because the biology changed. Not because the analysis logic was wrong. Not because the developer did poor work.

Most failures happen for a much more practical reason.

The **pipeline environment, configuration, or container dependency** quietly drifted.

A 2023 reproducibility benchmark of more than 1,200 Snakemake and Nextflow workflows reported that **72% of published bioinformatics pipelines failed to run on another machine**. The most common causes were not statistical errors or biological assumptions. They were infrastructure failures:

- Missing configuration files
- Conda environment resolution failures
- Broken or outdated containers

That matters because reproducibility is no longer just a “best practice” issue. It affects whether a lab can rerun its analysis, validate findings, transfer workflows to collaborators, or defend results in a publication.

The good news: the technical fixes now exist.

The problem: most custom pipelines still don't use them.

The real reproducibility problem

When people talk about reproducibility in bioinformatics, the discussion often jumps to documentation, GitHub repositories, or “using containers.”

Those help, but they are not enough.

A pipeline usually fails in one of three places:

1. **The configuration layer** The pipeline cannot find the right parameters, reference genome, sample sheet, or execution profile.
2. **The environment layer** Conda resolves a different dependency set than it did when the pipeline was first written.
3. **The container layer** The image tag changes, disappears, or no longer matches the hardware architecture.

Each layer needs a different fix.

A reproducible pipeline is not just a pipeline with code. It is code plus validated configuration, locked environments, and immutable containers.

Layer 1: Fix missing configs before the pipeline starts

The most common failure is also the least glamorous.

A pipeline runs on the developer’s machine because all the hidden files are there:

- Parameter files
- Reference genome paths
- Cluster profiles
- Bioconda channel settings
- Local execution assumptions
- Sample sheet formatting rules

Then someone else clones the repository, runs the workflow, and it fails before the first read is aligned.

Or worse, it runs using the wrong reference.

That is not a biology problem. That is a configuration problem.

The fix: Geniac + nextflow_schema.json

For Nextflow workflows, the configuration layer should be treated as a formal part of the pipeline.

Two tools help here:

1. Geniac

Geniac helps generate execution profiles for environments such as:

- Local execution
- SLURM
- SGE
- Kubernetes

Instead of leaving users to reconstruct profiles manually, the pipeline ships with profiles that match the intended execution environment.

2. nextflow_schema.json + nf-schema

The schema layer defines every expected input:

- Required parameters
- Reference paths
- Sample sheet columns
- Boolean flags
- Allowed values
- Default values

The key advantage is that validation happens before execution.

If the user gives a missing reference path, malformed sample sheet, or invalid parameter, the workflow fails immediately with a clear message.

That is much better than discovering the problem after two hours of compute time.

Why this works

Configuration becomes a declared artefact, not hidden knowledge.

In many custom pipelines, the developer knows what files are needed, but the pipeline itself does not enforce them. Schema validation changes that.

Before fixing containers or environments, this is the first layer to implement.

If a pipeline has no nextflow_schema.json, start there.

Layer 2: Stop letting Conda solve environments at runtime

Conda is useful, but it is not fully deterministic.

An environment.yml tells Conda what packages you want. It does not always guarantee the exact package versions and sub-dependencies that will be selected months later.

That means the same file can produce different environments depending on:

- Time
- Channel state
- Solver version
- Platform

- Sub-dependency updates

For bioinformatics, this matters because Bioconda moves fast. A workflow that installs correctly today may fail later, or worse, install successfully with slightly different versions.

The fix: Pixi + Wave / Seqera Containers

The principle is simple:

Do not let Conda resolve environments during production workflow execution.

1. Pixi for locked development environments

Pixi uses the Conda ecosystem but treats lockfiles as first-class.

A `pixi.lock` file records exact resolved versions across platforms such as:

- linux-64
- osx-arm64
- Other supported target systems

This means another user can install the same environment instead of asking Conda to resolve it again from scratch.

2. Wave / Seqera Containers for execution

Wave can turn an environment definition into an immutable container image.

Instead of the workflow depending on a live Conda solve, the environment is built once and used as a container.

The pipeline then references the container, not the raw `environment.yml`.

Why this works

Conda is moved out of the runtime path.

Pixi locks the environment for development. Wave materialises the environment as a container for execution.

That removes solver drift from the production workflow.

A practical rule:

Use Pixi for development. Use Wave-generated containers for execution. Do not run `conda env create` inside a production workflow.

Layer 3: Fix container rot with digest-pinned images

Containers are often treated as the final answer to reproducibility.

They are not.

A container only helps if it remains available and immutable.

Many workflows still reference images like this:

```
biocontainers/samtools:1.17
```

That looks stable, but it is still a tag.

Tags can move. Tags can disappear. Registries can deprecate images. Images may not support a new architecture.

That is container rot.

The fix: Wave freeze mode + private registry mirror

Wave freeze mode helps solve this by doing three important things.

1. Pin by digest, not tag

A digest is content-addressed.

That means the workflow points to the exact image bytes, not a label that can change.

Tags are names. Digests are content.

For reproducibility, content wins.

2. Mirror images to a private registry

If the upstream registry changes, the lab should not lose access to the image.

Mirroring makes the image part of the lab's controlled infrastructure instead of a borrowed external dependency.

3. Build for multiple architectures

This matters more now because workflows may move between:

- On-prem x86 clusters
- AWS Graviton
- ARM systems
- Apple Silicon development machines

A reproducible pipeline should not silently depend on one architecture unless that limitation is clearly declared.

Why this works

The container becomes a stable asset.

The lab is no longer relying on a mutable upstream tag or a public registry staying unchanged forever.

For long-term reproducibility, containers should be pinned by digest and mirrored.

The 2026 reproducibility stack

The order matters.

Do not start by containerising a poorly configured workflow.

Build the stack in this order:

```
Pipeline code
|
|-- Configuration layer
|   Geniac
|   nextflow_schema.json
|   nf-schema validation
|
|-- Environment layer
|   Pixi lockfiles
|   Wave / Seqera Containers
|
|-- Container layer
|   Digest-pinned images
|   Wave freeze mode
|   Private registry mirror
```

In simpler terms

Failure mode	Fix	Outcome
Missing configs	Geniac + nextflow_schema.json	Inputs are validated before execution
Conda solver drift	Pixi + Wave containers	Environments become locked and reproducible
Container rot	Digest pinning + private mirror	Images remain available and immutable

Each layer protects a different failure point.

Skip one layer, and that failure mode returns.

What this solves

This stack solves **infrastructure reproducibility**.

It helps answer:

- Will this pipeline run on another machine?
- Will it run six months later?
- Will another user get the same software environment?
- Will the required inputs be validated before execution?
- Will the container image still exist?

That is a major improvement for research groups, core facilities, and clinical-adjacent teams that need workflows to survive beyond one laptop or one PhD project.

What this does not solve

This stack does not guarantee **biological correctness**.

That distinction is important.

A schema can confirm that a reference genome path exists. It cannot confirm that the reference genome is the right biological choice.

Pixi can lock the DESeq2 version. It cannot confirm that DESeq2 is appropriate for the experimental design.

Wave can freeze the aligner container. It cannot decide whether STAR, HISAT2, Salmon, or another tool is the best choice for the dataset.

Reproducible execution and correct biological interpretation are related, but they are not the same.

Omics analysis can fail in two ways:

1. The pipeline fails to run.
2. The pipeline runs, but the analysis answers the wrong biological question.

This article focused on the first problem.

The second problem is harder, and it is where domain expertise still matters.

Final thought

Many labs do not need a completely new pipeline.

They need their existing pipeline to stop depending on hidden configuration, unstable Conda solves, and mutable container tags.

The practical starting point is clear:

1. Add schema validation.
2. Lock the environment.
3. Freeze and mirror the containers.

That is the difference between a workflow that works once and a workflow that can be trusted again six months later.

— **Parth Doshi** Bioinformatics Consultant Pipelines · Multiomics Integration · Downstream Analysis · Biostatistics

Sources

1. Grayson et al., 2023, ACM REP — *Automatic Reproduction of Workflows in the Snakemake Workflow Catalog and nf-core Registries*
2. Seqera Labs — Wave / Seqera Containers documentation
3. [prefix.dev](#) — Pixi package manager and lockfile documentation
4. Allain et al., 2022 — Geniac: Automatic Configuration GENERator and Installer for Nextflow pipelines

5. nf-core / Nextflow — nextflow_schema.json and nf-schema documentation

Enjoyed this article?

Subscribe to never miss an issue.



Omics Unlayered

Where omics quietly breaks — and how to catch it.

665 subscribers

[Subscribe](#)